

Data Encapsulation and De-Encapsulation

- Detailed notes

1. Introduction

In computer networks, communication between devices involves the transfer of data from one system to another.

However, this communication is not as simple as sending plain data — it requires **specific structures, protocols, and addressing mechanisms** to ensure the data reaches the correct destination safely and accurately.

To achieve this, the concept of **Data Encapsulation and De-Encapsulation** is used.

These processes occur across the **OSI (Open Systems Interconnection)** or **TCP/IP** model layers.

2. What is Data Encapsulation?

Definition:

Data Encapsulation is the process of **wrapping or packaging data** with the necessary **protocol information** before transmission across a network.

Each layer of the OSI or TCP/IP model adds its **own header (and sometimes a trailer)** to the data received from the upper layer.

This additional information helps the receiving device understand how to interpret and process the data.

3. Purpose of Data Encapsulation

- To ensure **reliable communication** between devices.

- To provide **layer-wise functionality** — each layer performs a specific role (e.g., addressing, routing, error detection).
- To help in **data segmentation** and **reassembly**.
- To maintain **protocol independence** — each layer can function without needing details of the others.
- To make network communication **structured and modular**.

4. The Process of Data Encapsulation

When a message is transmitted from a sender to a receiver, the data passes through **each layer of the OSI model**, where new headers are added step by step.

This process starts at the **Application Layer** and ends at the **Physical Layer**.

Step-by-Step Process of Data Encapsulation

Layer (OSI Model)	Data Unit	What Happens
1. Application Layer	Data	The user creates data (e.g., typing a message, sending an email, requesting a file). Protocols like HTTP, SMTP, FTP, etc., are involved here.
2. Presentation Layer	Data	Responsible for data translation, compression, and encryption . It ensures data is in a readable format for the application.
3. Session Layer	Data	Manages sessions and connections between sender and receiver — opens, maintains, and closes sessions.
4. Transport Layer	Segment	Divides data into segments and adds a transport header containing source and destination port numbers , sequence numbers, and control flags (e.g., TCP or UDP).
5. Network Layer	Packet / Datagram	Adds an IP header containing source and destination IP addresses for logical addressing and routing.
6. Data Link Layer	Frame	Encapsulates the packet into a frame by adding a frame header (containing MAC addresses) and a trailer (FCS – Frame Check Sequence) for error detection.
7. Physical Layer	Bits	Converts the frame into binary bits (0s and 1s) and transmits them over the physical medium (cables, radio waves, etc.).

Example of Data Encapsulation

Let's take a message "HELLO" sent from one computer to another:

1. **Application Layer:** Creates data "HELLO".
2. **Transport Layer:** Adds TCP header → [TCP Header][HELLO]
3. **Network Layer:** Adds IP header → [IP Header] [TCP Header] [HELLO]
4. **Data Link Layer:** Adds Frame header & trailer → [Frame Header] [IP Header] [TCP Header][HELLO] [Frame Trailer]
5. **Physical Layer:** Converts the entire frame to bits and sends it as electrical or optical signals.

5. What is De-Encapsulation?

Definition:

Data De-Encapsulation is the **reverse process of encapsulation**.

It occurs at the receiving device, where the encapsulated data moves **up the OSI model** from the **Physical Layer to the Application Layer**.

At each layer, the corresponding **header and trailer are removed**, and the data is passed to the upper layer for processing.

6. Process of Data De-Encapsulation

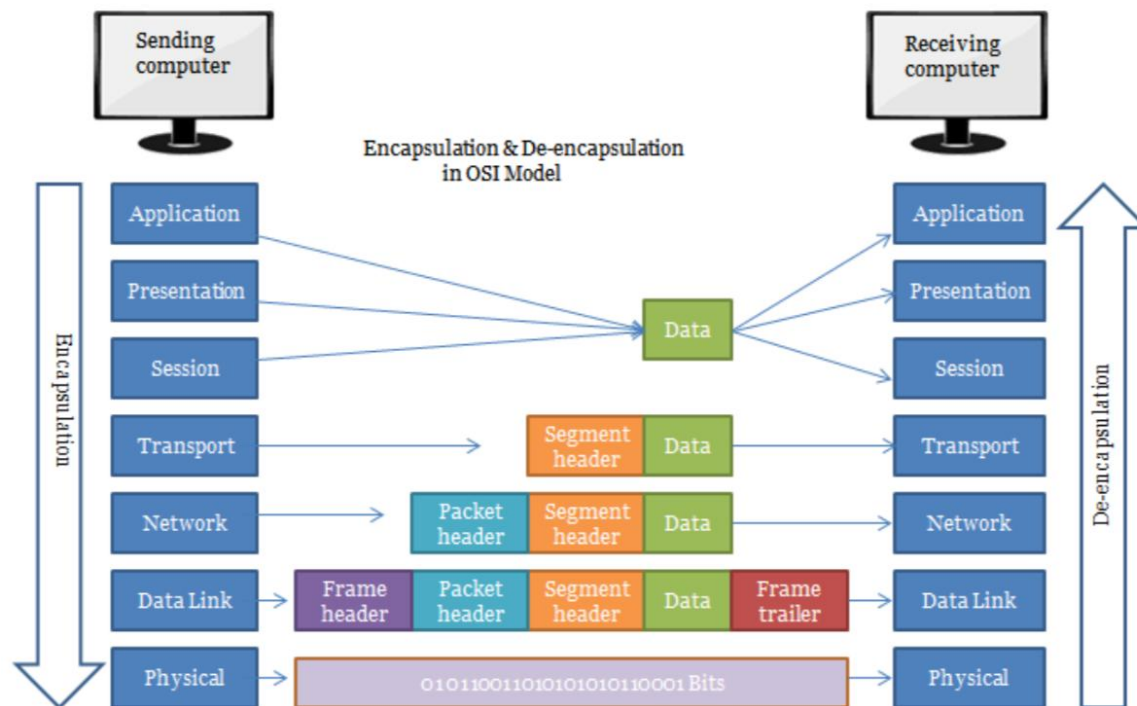
As data is received, each layer performs the reverse of what the sender did during encapsulation.

Layer (Receiving Side)	Process	Purpose
1. Physical Layer	Converts bits (signals) back into frames.	Responsible for receiving raw data from the medium.
2. Data Link Layer	Removes Frame Header and Trailer.	Checks for transmission errors using FCS and identifies the upper layer protocol.
3. Network Layer	Removes IP Header.	Examines source and destination IPs; determines whether the data is for this device.
4. Transport Layer	Removes TCP/UDP Header.	Uses port numbers to deliver the segment to the correct application process.
5. Session Layer	Manages and terminates sessions.	Ensures communication session is properly closed or maintained.
6. Presentation Layer	Decrypts and translates data.	Converts data into a human-readable or application-compatible format.
7. Application Layer	Receives the original message.	Presents final data to the user or application.

Example of De-Encapsulation

1. **Physical Layer:** Receives binary bits.
2. **Data Link Layer:** Converts bits into frames, checks for errors.
3. **Network Layer:** Reads the IP header and confirms destination.
4. **Transport Layer:** Uses port number to deliver data to the correct application.
5. **Application Layer:** Displays the message "HELLO" to the user.

7. Representation Diagram



8. Importance of Encapsulation and De-Encapsulation

- Ensures **data integrity and structure** during transmission.
- Allows **different network protocols** to communicate effectively.
- Provides **modularity** — each layer performs a specific role independently.
- Enables **error detection and correction** mechanisms.
- Makes **troubleshooting** easier (errors can be traced to specific layers).
- Maintains **security** through encryption at certain layers.

9. Key Differences Between Encapsulation and De-Encapsulation

Aspect	Encapsulation	De-Encapsulation
Definition	Wrapping data with headers/trailers before transmission.	Removing headers/trailers after reception.
Direction	From Application Layer → Physical Layer	From Physical Layer → Application Layer
Purpose	To prepare data for transmission.	To extract and interpret received data.
Performed by	Sender	Receiver
Adds / Removes	Adds information	Removes information
Data Flow	Down the OSI model	Up the OSI model

10. Real-World Example

When you send an email:

1. Your email text is created at the **Application Layer (SMTP)**.
2. It's segmented and assigned port numbers at the **Transport Layer (TCP)**.
3. The IP address of the receiver is added at the **Network Layer (IP)**.
4. The frame header and trailer are added at the **Data Link Layer (Ethernet)**.
5. Finally, the bits are sent via **Physical Layer (e.g., fiber, Wi-Fi)**.

At the receiver's side, each layer removes its respective headers and processes the message until it reaches the **mail application**, displaying your email

Encapsulation and De-Encapsulation in TCP/IP Model

1. Introduction

- The **TCP/IP model** is a simplified version of the OSI model used in real-world networking.
- It has **four layers**:
 - **Application Layer**
 - **Transport Layer**
 - **Internet Layer**
 - **Network Access Layer**
- **Encapsulation**: Adding headers (and sometimes trailers) as data moves **from top to bottom** (sender side).
- **De-Encapsulation**: Removing those headers as data moves **from bottom to top** (receiver side).

2. Encapsulation Process (Sender Side)

Data moves **down** the layers, and each layer **adds its own information**.

a) Application Layer

- User data is created (e.g., message, email, web request).
- Protocols: **HTTP, FTP, SMTP, DNS**.
- Data is just the **message** (no headers yet).

b) Transport Layer

- Breaks data into **segments** (TCP) or **datagrams** (UDP).
- Adds a **transport header** that includes:
 - Source and destination **port numbers**
 - **Sequence numbers** (for data order)

- o **Error checking** info
- Protocols: **TCP** (reliable), **UDP** (fast).
- Now data = **Segment**

c) Internet Layer

- Adds **IP header** with:
 - o **Source and destination IP addresses**
 - o **Routing information**
- Protocols: **IP (IPv4/IPv6)**, **ICMP**, **ARP**.
- Now data = **Packet**

d) Network Access Layer

- Adds **MAC addresses** and **error detection trailer (FCS)**.
- Converts the data into **bits (0s and 1s)** for transmission.
- Protocols: **Ethernet**, **Wi-Fi**.
- Now data = **Frame** → **Bits**

3. De-Encapsulation Process (Receiver Side)

Data moves **up** the layers, and each layer **removes its own header**.

a) Network Access Layer

- Receives the raw bits (signals).
- Reconstructs **frames** and checks for errors using FCS.
- Removes the frame header and trailer.

b) Internet Layer

- Reads and removes the **IP header**.
- Identifies the **source and destination IPs**.
- Delivers the packet to the correct transport protocol (TCP/UDP).

c) Transport Layer

- Removes the **transport header** (port numbers, sequence info).
- Reassembles segments into the original data stream.
- Sends data to the correct application process.

d) Application Layer

- Receives the final, original message.
- Delivers it to the user (e.g., shows webpage, email, or message).

4. Data Flow in TCP/IP Model

Encapsulation (Sender):

Application → Transport → Internet → Network Access
Data → Segment → Packet → Frame → Bits

De-Encapsulation (Receiver):

Network Access → Internet → Transport → Application
Bits → Frame → Packet → Segment → Data